

Parte 6: Laboratorio 6: Telemetría P4 — Inspección Multi-Salto de Rutas y Medición de Latencia In-Switch

Validación académica – MRI (Multi-Hop Route Inspection) y MySec (latencia en el plano de datos)

Convención de colores

A lo largo de este documento se usa la misma convención visual de las validaciones previas para distinguir el origen de cada fragmento:

- **Verde**: comandos ejecutados en el shell Bash de la máquina virtual.
- **Azul**: fragmentos de scripts Python (topología, send/receive).
- **Rojo**: comandos ejecutados desde la CLI de Mininet o en terminales de hosts virtuales.
- **Naranja**: fragmentos de código P4 directamente relevantes para el comportamiento validado.

Introducción

Este documento constituye la validación académica del sexto laboratorio de la secuencia de prácticas de redes programables del curso Internet del Futuro. El laboratorio introduce dos técnicas de telemetría de red implementadas íntegramente en el plano de datos mediante P4: la Inspección Multi-Salto de Rutas (MRI, variante simplificada de In-Band Network Telemetry) y la medición de latencia in-switch mediante el protocolo personalizado MySec [6, 1].

Ambos ejercicios comparten el mismo flujo de trabajo de compilación e implantación establecido en los laboratorios anteriores: compilación con p4c-bm2-ss, arranque de la topología con topo.py y carga de reglas de reenvío mediante

simple_switch_CLI. La diferencia fundamental respecto a los laboratorios previos reside en que los paquetes ya no se limitan a portar datos de usuario: se convierten en portadores activos de información de medición que los propios switches escriben a velocidad de línea, sin intervención del plano de control durante la fase de medición.

El ejercicio guiado (MRI) implementa una variante simplificada de INT. Cada switch que atraviesa un paquete agrega su propio identificador y la profundidad de cola de salida a una pila de cabeceras creciente transportada dentro del paquete como opción IPv4. El receptor puede leer la secuencia completa de saltos y el estado de congestión por salto directamente en el paquete que llega, sin consultar ningún sistema de monitoreo externo [3].

El ejercicio de actividad del estudiante (MySec) implementa un protocolo personalizado de medición de latencia in-switch. Un paquete construido por el emisor sigue un camino de rebote ($h1 \rightarrow s1 \rightarrow s2 \rightarrow s1 \rightarrow h1$) mientras los switches anotan marcas de tiempo en campos dedicados de la cabecera. Al regresar a $h1$, el emisor lee los valores de latencia del propio paquete de retorno, sin coordinación fuera de banda. Una distinción clave respecto a MRI es que MySec utiliza el número de protocolo IP 169 (valor no asignado por IANA, empleado aquí con fines pedagógicos) para activar la lógica del plano de datos, mientras que MRI se apoya en el tipo de opción IPv4 con número experimental 31 (valor no registrado por IANA, elegido por los autores del tutorial siguiendo el esquema de codificación de RFC 791 [5]) [8].

La diferencia conceptual central entre los dos ejercicios es el tipo de información recolectada y el punto del pipeline donde ocurre: MRI obtiene trazas de ruta y estado de congestión en el pipeline de egreso mediante el metadato de profundidad de cola de la arquitectura v1model, mientras MySec mide la latencia de procesamiento del switch en el pipeline de egreso usando los metadatos de marca de tiempo global disponibles en v1model. Ambos ejercicios ilustran que, mediante operaciones como push_front sobre pilas de cabeceras y lectura de metadatos internos (deq_qdepth, timestamps globales), el mismo pipeline que reenvía paquetes puede instrumentarse para exponer ruta, congestión y latencia sin recurrir a sondas externas ni ASICs especializados [1, 2].

Ejercicio 1: MRI — Inspección Multi-Salto de Rutas (Guiado)

Contexto teórico: INT y MRI

In-Band Network Telemetry (INT) es un marco definido por el consorcio P4.org que permite a los dispositivos de red embeber datos de monitoreo directamente en los paquetes del plano de datos a velocidad de línea, sin generar mensajes de control separados ni copiar paquetes hacia un sistema externo [6]. En su forma completa, INT transporta información por salto como identificador de switch, profundidad de cola, marcas de tiempo de ingreso y egreso, y utilización de enlace. Los paquetes que portan datos INT acumulan esta información en un stack de cabeceras que crece con cada salto; el receptor final extrae y reporta los datos sin interacción con el plano de control.

La especificación INT de P4.org distingue tres modos de operación: INT-MD (Metadata), el modo original donde los metadatos se acumulan en el propio paquete de datos en forma de una pila creciente; INT-XD (eXport Data), donde los metadatos se extraen al final del camino y se exportan como informes separados; e INT-MX (eMbedded eXtension), donde los metadatos se incrustan en encabezados de extensión dedicados. La especificación también define los roles de INT-Source (primer switch de la ruta, inserta la cabecera INT vacía), INT-Transit (cada switch intermedio que agrega su registro) e INT-Sink (último switch, extrae los datos y los reenvía al colector) [4].

MRI (Multi-Hop Route Inspection) es la versión simplificada utilizada en este laboratorio, adaptada del repositorio oficial de tutoriales de P4.org [3]. En lugar de implementar el stack completo INT-MD con todos los metadatos por salto, MRI se limita a dos campos: el identificador del switch (swid) y la profundidad de cola de egreso (qdepth). Esta reducción permite concentrarse en los mecanismos fundamentales de P4 involucrados: parsers recursivos con estado dependiente de datos, pilas de encabezados con `push_front`, tablas sin clave de coincidencia instaladas con `table_set_default`, modificación dinámica de campos de longitud del encabezado IPv4 en egreso, y recálculo del checksum. Todos estos mecanismos se usan conjuntamente en sistemas reales de

telemetría INT en entornos de centros de datos [4, 9].

A diferencia de INT completo, MRI usa Opciones IPv4 como mecanismo de transporte en lugar de encabezados de extensión dedicados. El número de opción experimental 31 (no asignado por IANA, definido en el tutorial siguiendo el esquema de codificación de RFC 791 [5]) señala la presencia del encabezado MRI. Cada switch que procesa el paquete en el pipeline de egreso llama a `push.front(1)` para hacer crecer la pila en un registro, escribe su propio identificador y la profundidad de cola de salida, e incrementa los campos `ihl`, `totalLen` y `optionLength` en consecuencia. El receptor lee la pila completa, que lista los switches en orden inverso al camino (el último salto primero), obteniendo tanto la ruta que tomó el paquete como el estado de congestión en cada nodo [1].

Descripción de la topología y adaptaciones del ejercicio

El ejercicio está basado en el ejercicio `mri` del repositorio oficial `p4lang/tutorials` [3]. La adaptación más significativa es la migración de la infraestructura de control: el ejercicio original utiliza `simple_switch_grpc` como target BMv2 y archivos JSON en formato P4Runtime para las reglas de reenvío; esta adaptación emplea `simple_switch` y la CLI Thrift (`simple_switch_CLI`), eliminando completamente la dependencia de gRPC, protobuf y la librería cliente P4Runtime. Los archivos `s1-runtime.json`, `s2-runtime.json` y `s3-runtime.json` fueron reemplazados por `s1-commands.txt`, `s2-commands.txt` y `s3-commands.txt` en formato de comandos CLI. El script `Makefile` original fue eliminado; la compilación y el arranque de la topología se realizan manualmente. Los scripts `send.py` y `receive.py` se conservan del repositorio original sin modificaciones [3].

Se mantuvieron los tres switches del diseño original porque son fundamentales para el concepto de Multi-Hop Route Inspection: con un solo switch siempre habría exactamente un registro en la pila MRI, y con dos switches el camino sería siempre único y fijo. Con tres switches existen múltiples rutas posibles ($h1 \rightarrow h2$ cruza $s1-s2$; $h1 \rightarrow h3$ cruza $s1-s3$), y el header stack refleja qué par de switches procesó cada paquete, haciendo real la inspección

de ruta. Ningún paquete individual en esta topología pasa por los tres switches simultáneamente; siempre hay exactamente dos registros en la pila. Lo que varía entre flujos es qué par de switches aparece: la ruta $h1 \rightarrow h2$ produce $\{\text{swid}=1, \text{swid}=2\}$, la ruta $h1 \rightarrow h3$ produce $\{\text{swid}=1, \text{swid}=3\}$, y la ruta $h2 \rightarrow h3$ produce $\{\text{swid}=2, \text{swid}=3\}$.

La topología consiste en tres switches P4 y cinco hosts. Dos hosts están conectados a s1 (subred 10.0.1.0/24), dos a s2 (subred 10.0.2.0/24) y uno a s3 (subred 10.0.3.0/24). El enlace entre s1 y s2 está limitado en tasa a 0,5 Mbps, actuando como cuello de botella que puede saturarse con tráfico de fondo para producir valores de profundidad de cola observables en los registros MRI. La asignación de puertos es la siguiente:

- s1 (puerto Thrift 9090): h1 en 10.0.1.1/24 en puerto 1, h11 en 10.0.1.11/24 en puerto 2, s2 en puerto 3 (cuello de botella, 0,5 Mbps), s3 en puerto 4.
- s2 (puerto Thrift 9091): h2 en 10.0.2.2/24 en puerto 1, h22 en 10.0.2.22/24 en puerto 2, s1 en puerto 3, s3 en puerto 4.
- s3 (puerto Thrift 9092): h3 en 10.0.3.3/24 en puerto 1, s1 en puerto 2, s2 en puerto 3.

El ejercicio usa reenvío de capa 3 (LPM sobre IPv4 destino). Cada host está configurado con una ruta por defecto hacia una IP de gateway virtual, y el switch reescribe la MAC Ethernet destino en cada salto. Las entradas ARP estáticas son pre-configuradas en cada host por topo.py, eliminando la resolución ARP como variable en el experimento [2].

Paso 1 – Compilación del programa P4

El primer paso es hacer ejecutables los scripts auxiliares y compilar el programa P4. El compilador p4c-bm2-ss transforma el programa P4-16 en una representación intermedia JSON que describe el pipeline completo: parser, controles de ingreso y egreso, cómputo de checksum y deparser [1, 2].

Desde el directorio del ejercicio MRI:

```
chmod +x send.py receive.py
```

```
mkdir -p p4src/build
```

```
p4c-bm2-ss --p4v 16 -o p4src/build/bmv2.json p4src/mri.p4
```

La salida observada fue la siguiente:

```
root@p4:/home/p4/P4_Labs_Docs/LabP4_3/Codigo/mri# mkdir -p p4src/build
root@p4:/home/p4/P4_Labs_Docs/LabP4_3/Codigo/mri# p4c-bm2-ss --p4v 16 -o
p4src/build/bmv2.json p4src/mri.p4
root@p4:/home/p4/P4_Labs_Docs/LabP4_3/Codigo/mri#
```

Una compilación exitosa sin mensajes de error confirma que el programa es sintácticamente correcto y que todas las partes del pipeline son reconocidas por p4c-bm2-ss.

Desde el punto de vista del diseño del pipeline, mri.p4 define las siguientes cabeceras específicas para MRI junto a las cabeceras estándar Ethernet e IPv4:

```
header ipv4_option_t {
    bit<1>  copyFlag;          // bit de copia para opciones IP
    bit<2>  optClass;          // clase: 0 = control
    bit<5>  option;            // valor 31 senala MRI
    bit<8>  optionLength;      // longitud total de la opcion en bytes
}

header mri_t {
    bit<16> count;             // numero de registros de switch en la pila
}

header switch_t {
    switchID_t swid;           // identificador de 32 bits del switch
    qdepth_t  qdepth;          // profundidad de cola de 32 bits en egreso
}

struct headers {
    // ...

    switch_t[MAX_HOPS] swtraces; // pila de registros, MAX_HOPS = 9
}
```

La pila `switch_t[MAX_HOPS]` es la estructura central de MRI. En P4, un `header stack` es un array de instancias del mismo tipo de cabecera cuyo tamaño máximo se fija en tiempo de compilación (aquí, 9), pero cuyo número de elementos válidos puede variar en tiempo de ejecución mediante las operaciones `push_front` y `pop_front` [1]. Esto permite que la pila crezca en cada salto sin modificar la definición del tipo de datos.

Paso 2 – Inicio de la topología Mininet

El script `topo.py` instancia la topología emulada mediante la API de Mininet. Para cada switch P4, el script invoca `simple_switch` con los argumentos de interfaz, puerto Thrift, `--device-id` y ruta al JSON compilado. Los hosts reciben sus direcciones IP, rutas por defecto y entradas ARP estáticas dentro del mismo script [2].

Dejar este terminal abierto durante toda la práctica:

```
sudo python3 mininet/topo.py
```

La salida observada en la práctica fue la siguiente:

```
*** Creating network
*** Adding hosts:
h1 h2 h3 h11 h22
*** Adding switches:
s1 s2 s3
*** Adding links:
(s1, h1) (s1, h11) (0.50Mbit) (0.50Mbit) (s1, s2) (s1, s3)
(s2, h2) (s2, h22) (s3, h3) (s3, s2)
*** Configuring hosts
h1 h2 h3 h11 h22
*** Starting controller

*** Starting 3 switches
s1 Starting s1 @ thrift-port 9090
  cmd: simple_switch -i 1@s1-eth1 -i 2@s1-eth2 -i 3@s1-eth3 -i 4@s1-eth4
      --thrift-port 9090 --device-id 0 ../p4src/build/bmv2.json
s2 Starting s2 @ thrift-port 9091
  cmd: simple_switch -i 3@s2-eth3 -i 1@s2-eth1 -i 2@s2-eth2 -i 4@s2-eth4
      --thrift-port 9091 --device-id 1 ../p4src/build/bmv2.json
s3 Starting s3 @ thrift-port 9092
  cmd: simple_switch -i 2@s3-eth2 -i 1@s3-eth1 -i 3@s3-eth3
      --thrift-port 9092 --device-id 2 ../p4src/build/bmv2.json
```


La salida confirma que Mininet creó la red con los cinco hosts y los tres switches, que el enlace cuello de botella s1-s2 tiene 0,50 Mbps de ancho de banda, y que cada switch arrancó su proceso simple_switch escuchando en el puerto Thrift correspondiente. Los argumentos `--device-id 0, 1 y 2` distinguen las instancias BMv2 y evitan colisiones en los sockets IPC que cada instancia crea en `/tmp/`. El mensaje Starting controller sin controlador real es un artefacto de la API de Mininet sin efecto práctico, ya que simple_switch no requiere controlador externo [2].

Paso 3 – Instalación de las reglas de reenvío

Una vez que los tres switches están en ejecución, es necesario poblar sus tablas de reenvío. Este ejercicio introduce un comando CLI nuevo respecto a los laboratorios anteriores: **table_set_default**. A diferencia de `table_add`, que instala una entrada de coincidencia identificada por sus campos de clave, `table_set_default` establece la acción que se ejecuta cuando ninguna entrada coincide o, como en este caso, cuando la tabla no tiene ningún campo de clave definido. La tabla `MyEgress.swtrace` no tiene campo `key`; ejecuta la acción `add_swtrace` de forma incondicional para cada paquete que llega al pipeline de egreso con el encabezado MRI válido. El argumento numérico (1, 2 o 3) es el identificador único que cada instancia BMv2 escribirá en el campo `swid` de cada nuevo registro MRI [1, 2].

En un segundo terminal, instalar las reglas en los tres switches:

```
simple_switch_CLI --thrift-port 9090 <s1-commands.txt
```

```
Obtaining JSON from switch...
Done
Control utility for runtime P4 table manipulation
RuntimeCmd: Setting default action of MyEgress.swtrace
action:           MyEgress.add_swtrace
runtime data:     00:00:00:01
RuntimeCmd: Adding entry to lpm match table MyIngress.ipv4_lpm
match key:        LPM-0a:00:01:01/32
action:           MyIngress.ipv4_forward
runtime data:     08:00:00:00:01:01  00:01
Entry has been added with handle 0
RuntimeCmd: Adding entry to lpm match table MyIngress.ipv4_lpm
match key:        LPM-0a:00:01:0b/32
```



```

action:          MyIngress.ipv4_forward
runtime data:    08:00:00:00:01:11  00:02
Entry has been added with handle 1
RuntimeCmd: Adding entry to lpm match table MyIngress.ipv4_lpm
match key:      LPM-0a:00:02:00/24
action:          MyIngress.ipv4_forward
runtime data:    08:00:00:00:02:00  00:03
Entry has been added with handle 2
RuntimeCmd: Adding entry to lpm match table MyIngress.ipv4_lpm
match key:      LPM-0a:00:03:00/24
action:          MyIngress.ipv4_forward
runtime data:    08:00:00:00:03:00  00:04
Entry has been added with handle 3

```

```
simple_switch_CLI --thrift-port 9091 <s2-commands.txt
```

```

Obtaining JSON from switch...
Done
Control utility for runtime P4 table manipulation
RuntimeCmd: Setting default action of MyEgress.swtrace
action:          MyEgress.add_swtrace
runtime data:    00:00:00:02
RuntimeCmd: Adding entry to lpm match table MyIngress.ipv4_lpm
match key:      LPM-0a:00:02:02/32
action:          MyIngress.ipv4_forward
runtime data:    08:00:00:00:02:02  00:01
Entry has been added with handle 0
RuntimeCmd: Adding entry to lpm match table MyIngress.ipv4_lpm
match key:      LPM-0a:00:02:16/32
action:          MyIngress.ipv4_forward
runtime data:    08:00:00:00:02:22  00:02
Entry has been added with handle 1
RuntimeCmd: Adding entry to lpm match table MyIngress.ipv4_lpm
match key:      LPM-0a:00:01:00/24
action:          MyIngress.ipv4_forward
runtime data:    08:00:00:00:01:00  00:03
Entry has been added with handle 2
RuntimeCmd: Adding entry to lpm match table MyIngress.ipv4_lpm
match key:      LPM-0a:00:03:00/24
action:          MyIngress.ipv4_forward
runtime data:    08:00:00:00:03:00  00:04
Entry has been added with handle 3

```

```
simple_switch_CLI --thrift-port 9092 <s3-commands.txt
```

```

Obtaining JSON from switch...
Done
Control utility for runtime P4 table manipulation
RuntimeCmd: Setting default action of MyEgress.swtrace

```

```

action:          MyEgress.add_swtrace
runtime data:    00:00:00:03
RuntimeCmd: Adding entry to lpm match table MyIngress.ipv4_lpm
match key:       LPM-0a:00:03:03/32
action:          MyIngress.ipv4_forward
runtime data:    08:00:00:00:03:03  00:01
Entry has been added with handle 0
RuntimeCmd: Adding entry to lpm match table MyIngress.ipv4_lpm
match key:       LPM-0a:00:01:00/24
action:          MyIngress.ipv4_forward
runtime data:    08:00:00:00:01:00  00:02
Entry has been added with handle 1
RuntimeCmd: Adding entry to lpm match table MyIngress.ipv4_lpm
match key:       LPM-0a:00:02:00/24
action:          MyIngress.ipv4_forward
runtime data:    08:00:00:00:02:00  00:03
Entry has been added with handle 2

```

La primera línea de cada bloque CLI confirma la instalación del identificador de switch: runtime data 00:00:00:01 para s1, 00:00:00:02 para s2 y 00:00:00:03 para s3. Estos valores son los parámetros de la acción add_swtrace que se escribe en el campo swid de cada registro MRI.

Las entradas LPM configuran el reenvío L3: s1 y s2 tienen cuatro entradas cada uno (dos /32 para sus hosts locales y dos /24 para las subredes remotas), mientras s3 tiene tres entradas (una /32 para h3 y dos /24 para las subredes de s1 y s2). La clave en formato LPM (0a:00:01:01/32) es la representación hexadecimal de la dirección IP 10.0.1.1 con máscara /32; 0a:00:01:0b es 10.0.1.11 y 0a:00:02:00/24 es la ruta agregada 10.0.2.0/24. La acción ipv4_forward toma como parámetros la MAC del siguiente salto y el puerto de egreso en hexadecimal de dos bytes (00:01, 00:02, 00:03, 00:04). En s3, la ruta 10.0.2.0/24 sale por el puerto 3 (hacia s2) con MAC de gateway 08:00:00:00:02:00; la ruta 10.0.1.0/24 sale por el puerto 2 (hacia s1) con MAC 08:00:00:00:01:00.

Parser recursivo y acción add_swtrace

El parser de mri.p4 implementa un mecanismo de análisis recursivo para consumir un número variable de registros switch_t. Este es el primer ejercicio de la secuencia de laboratorios que requiere un parser con estado dependiente de datos (data-dependent parsing). El estado parse_mri extrae el encabe-

zando base MRI y almacena el contador en metadatos del parser; el estado `parse_swtrace` extrae un registro por invocación y se autoinvoca hasta que el contador llega a cero:

```
state parse_mri {
    packet.extract(hdr.mri);
    meta.parser_metadata.remaining = hdr.mri.count;
    transition select(meta.parser_metadata.remaining) {
        0      : accept;
        default: parse_swtrace;
    }
}

state parse_swtrace {
    packet.extract(hdr.swtraces.next);
    meta.parser_metadata.remaining =
        meta.parser_metadata.remaining - 1;
    transition select(meta.parser_metadata.remaining) {
        0      : accept;
        default: parse_swtrace; // recursion
    }
}
```

La recursión es necesaria porque el número de registros en la pila es variable y se conoce solo en tiempo de ejecución a partir del campo `count`. La referencia `hdr.swtraces.next` es una sintaxis especial de pilas de encabezados P4 que referencia la primera posición inválida en la pila y avanza automáticamente el puntero interno en cada llamada a `packet.extract` [1].

La acción `add_swtrace` en el control de egreso es el núcleo del comportamiento MRI. Se ejecuta en el pipeline de egreso, no en ingreso. Esto es fundamental: el metadato `standard_metadata.deq_qdepth` (profundidad de cola en el momento de la dequeue del paquete) solo está disponible en egreso, porque refleja el estado de la cola de salida después de que el paquete ha sido seleccionado para transmisión [1, 2]:

```
action add_swtrace(switchID_t swid) {
```

```

hdr.mri.count          = hdr.mri.count + 1;
hdr.swtraces.push_front(1); // hace crecer la pila
hdr.swtraces[0].setValid();
hdr.swtraces[0].swid   = swid;
hdr.swtraces[0].qdepth =
    (qdepth_t) standardmetadata.deq_qdepth;
hdr.ipv4.ihl           = hdr.ipv4.ihl + 2;
hdr.ipv4.option.optionLength =
    hdr.ipv4.option.optionLength + 8;
hdr.ipv4.totalLen      = hdr.ipv4.totalLen + 8;
}

```

La llamada `push_front(1)` desplaza todos los registros existentes una posición hacia adelante en la pila y deja la posición 0 libre (invalidada). El `setValid()` activa la nueva celda, y las siguientes asignaciones escriben el identificador del switch y la profundidad de cola. Dado que cada registro agrega 8 bytes (4 bytes para `swid` + 4 bytes para `qdepth`), el campo `ihl` de IPv4 se incrementa en 2 (unidades de 4 bytes), `optionLength` en 8, y `totalLen` en 8. Estos tres incrementos mantienen coherencia entre el tamaño declarado del paquete y su contenido real; sin ellos, los receptores descartarían los paquetes por inconsistencia de longitud. Finalmente, el bloque `MyComputeChecksum` recalcula el checksum IPv4 con los valores actualizados [5].

Paso 4 – Verificación de conectividad

Con las tablas pobladas, se verifica la conectividad mediante el comando `pingall` de Mininet:

```

mininet> pingall

*** Ping: testing ping reachability
h1 -> h2 h3 X h22
h2 -> h1 h3 h11 X
h3 -> h1 h2 h11 h22
h11 -> X h2 h3 h22
h22 -> h1 X h3 h11
*** Results: 20% dropped (16/20 received)

```

El resultado del 20 % de paquetes descartados es el comportamiento esperado y correcto. Los cuatro pares que fallan son h1–h11 (ambos en s1, subred 10.0.1.0/24) y h2–h22 (ambos en s2, subred 10.0.2.0/24), en ambas direcciones. Dado que estos pares comparten la misma subred /24, la pila de red Linux de cada host intenta resolver la dirección IP destino mediante ARP directamente, sin enrutar a través del gateway. El switch P4 no responde a solicitudes ARP, por lo que la resolución falla. Este comportamiento es correcto y esperado: confirma que el modelo de reenvío implementado (L3 entre subredes, sin manejo ARP intra-subred) es exactamente el diseñado. Todos los pares entre subredes distintas funcionan correctamente, validando que las rutas LPM y la reescritura de MACs operan como se espera [2].

Paso 5 – Prueba MRI con congestión inducida

Esta es la etapa central del ejercicio. El objetivo es observar cómo el header stack MRI crece a medida que el paquete atraviesa switches, y cómo los valores de profundidad de cola varían cuando el enlace cuello de botella s1–s2 está saturado.

Abrir terminales adicionales para los hosts desde la CLI de Mininet:

```
mininet> xterm h1 h2 h11 h22
```

En el terminal de h22, iniciar un servidor UDP para absorber el flujo de fondo:

```
h22# iperf -s -u
```

En el terminal de h2, iniciar el receptor MRI basado en Scapy:

```
h2# ./receive.py
```

En el terminal de h11, iniciar el flujo de fondo saturante hacia h22:

```
h11# iperf -c 10.0.2.22 -t 15 -u
```

Inmediatamente después, en el terminal de h1, enviar paquetes etiquetados con MRI hacia h2:

```
h1# ./send.py 10.0.2.2 "P4 is cool"30
```

El script send.py construye paquetes UDP que incluyen una opción IPv4 de tipo 31 con count=0 y una lista swtrace vacía. Esto señala a cada switch en el camino que el paquete porta datos MRI. La salida observada en h2 durante los primeros paquetes (antes de la congestión) fue la siguiente:

```
sniffing on eth0
got a packet
###[ Ethernet ]###
  dst      = 08:00:00:00:02:02
  src      = 08:00:00:00:02:00
  type     = IPv4
###[ IP ]###
  version  = 4
  ihl      = 10
  tos      = 0x0
  len      = 58
  id       = 1
  flags    =
  frag     = 0
  ttl      = 62
  proto    = udp
  chksum   = 0x60b0
  src      = 10.0.1.1
  dst      = 10.0.2.2
  \options \
    |###[ MRI ]###
    | copy_flag = 0
    | optclass  = control
    | option    = 31
    | length    = 20
    | count     = 2
    | \swtraces \
    |   |###[ SwitchTrace ]###
    |   | swid      = 2
    |   | qdepth    = 1
    |   |###[ SwitchTrace ]###
    |   | swid      = 1
    |   | qdepth    = 0
###[ UDP ]###
  sport    = 1234
  dport    = 4321
  len      = 18
  chksum   = 0x1c7b
###[ Raw ]###
  load     = 'P4 is cool'
```

Una vez que el flujo iperf de h11 satura el enlace s1–s2, los valores de qdepth para swid=1 aumentan significativamente:

```

got a packet
###[ IP ]###
  ihl      = 10
  ttl      = 62
  proto    = udp
  src      = 10.0.1.1
  dst      = 10.0.2.2
  \options \
    |###[ MRI ]###
    | option   = 31
    | length   = 20
    | count    = 2
    | \swtraces \
    |   |###[ SwitchTrace ]###
    |   | swid      = 2
    |   | qdepth    = 1
    |   |###[ SwitchTrace ]###
    |   | swid      = 1
    |   | qdepth    = 61

```

El análisis de las capturas revela los siguientes puntos clave:

count = 2 confirma que exactamente dos switches procesaron el paquete (s1 y s2 en la ruta $h1 \rightarrow s1 \rightarrow s2 \rightarrow h2$). La pila está ordenada con el registro más reciente primero: swid=2 fue insertado por s2 (el último switch antes de h2), y swid=1 fue insertado por s1. Este orden inverso es consecuencia de push_front: cada switch inserta su registro al inicio de la pila, desplazando los anteriores.

Variación de qdepth bajo congestión: En el primer paquete los valores son bajos (0 y 1) porque el flujo de fondo no ha saturado el enlace todavía. En el paquete con congestión, qdepth de swid=1 subió a 61 paquetes en espera, reflejando la saturación del enlace s1–s2 provocada por el flujo iperf de h11. El qdepth de swid=2 permanece en 1, indicando que el enlace s2–h2 no está congestionado. Esta asimetría es exactamente la información diagnóstica que permite localizar el cuello de botella: el problema está en la cola de egreso de s1, no en s2 [4].

ihl = 10 (en lugar del valor mínimo 5) confirma que el encabezado IPv4 contiene opciones. El incremento de 5 a 10 representa 20 bytes adicionales: el encabezado ipv4_option_t (2 bytes: 1 byte de tipo de opción + 1 byte option-

Length), el encabezado `mri_t` (2 bytes para count) y dos registros `switch_t` de 8 bytes cada uno (4 bytes `swid` + 4 bytes `qdepth`), totalizando $2 + 2 + 16 = 20$ bytes. La acción `add_swtrace` calcula esto incrementando `ihl` en 2 (8 bytes / 4 bytes por unidad IHL) por cada registro que agrega, garantizando coherencia con el campo de longitud real del paquete [5].

TTL = 62 (decrementado desde 64 original) confirma que el paquete atravesó exactamente dos switches, y que la acción `ipv4_forward` decrementó el TTL en cada uno. Esto es consistente con la ruta $h1 \rightarrow s1 \rightarrow s2 \rightarrow h2$.

Paso 6 – Visibilidad de ruta en camino alternativo

La observación central de Multi-Hop Route Inspection no es solo la profundidad de cola, sino la capacidad de determinar qué switches procesaron el paquete. Este paso demuestra esa capacidad enviando tráfico hacia `h3`, que usa la ruta alternativa `s1-s3`.

```
mininet> xterm h3
```

```
h3# ./receive.py
```

```
h1# ./send.py 10.0.3.3 "P4 is cool"5
```

En este caso la pila MRI muestra `swid=3` (más reciente, insertado por `s3`) y `swid=1` (insertado por `s1`), con `count=2`. El contraste con la prueba anterior ($h1 \rightarrow h2$) es directo: el primer elemento del stack identifica al switch que procesó el paquete justo antes del destino. Esta distinción constituye precisamente la dimensión de inspección de ruta de MRI: el propio paquete porta la evidencia de su trayectoria. Un mismo flujo TCP entre `h1` y un destino puede ser redirigido a través de diferentes switches (por ejemplo, por balanceo de carga ECMP), y los registros MRI permiten detectar ese cambio de ruta sin ningún monitoreo externo. La ruta $h2 \rightarrow h3$ produce $\{\text{swid}=2, \text{swid}=3\}$, completando el conjunto de pares posibles en esta topología [4, 3].

Ejercicio 2: MySec — Medición de Latencia In-Switch (Actividad del estudiante)

Contexto teórico

La medición de latencia en el plano de datos es uno de los casos de uso más directos de los planos de datos programables. En redes de producción, la latencia de procesamiento de un switch ASIC es del orden de cientos de nanosegundos; en un switch software como BMv2 es del orden de microsegundos. En ambos casos, medir esta latencia sin instrumentación externa requiere que el propio switch registre marcas de tiempo antes y después del procesamiento del paquete.

La arquitectura v1model de P4 expone dos metadatos de marca de tiempo global relevantes para este ejercicio: `standard_metadata.ingress_global_timestamp`, que captura la marca de tiempo del momento en que el paquete llegó al switch (inicio del procesamiento de ingreso), y `standard_metadata.egress_global_timestamp`, que captura la marca de tiempo del momento en que el paquete inició el procesamiento de egreso. En la implementación BMv2 `simple_switch` (arquitectura v1model), ambos valores se expresan en microsegundos. La diferencia entre ambos en el pipeline de egreso aproxima el tiempo que el paquete permaneció en el switch (incluyendo espera en cola y procesamiento del pipeline de ingreso). Estos metadatos se actualizan y tienen significado completo en el pipeline de egreso; aunque `ingress_global_timestamp` se fija al inicio del procesamiento de ingreso, la latencia debe calcularse en egreso porque es allí donde `egress_global_timestamp` queda registrado y ambos valores son simultáneamente coherentes y comparables para la misma instancia de paquete [1, 2].

MySec implementa este principio de medición en un protocolo de rebote. En lugar de requerir dos switches con relojes sincronizados (como sería necesario en un escenario RTT tradicional entre dispositivos separados), el protocolo hace que el paquete vuelva al switch de origen, de modo que todas las mediciones de interés se realizan en el mismo dispositivo, eliminando el problema de sincronización de relojes. El protocolo usa el número de protocolo IP 169,

un valor sin asignación IANA, como selector en el parser e ingreso para distinguir los paquetes MySec del tráfico estándar. Solo los paquetes que portan un encabezado mysec_t válido activan la lógica de marcado de tiempo en el pipeline de egreso. El tráfico estándar (ping, TCP, iperf) usa números de protocolo diferentes y es enrutado por la tabla L2 ordinaria, sin interferir con la medición [8].

Desde el punto de vista del diseño P4, MySec usa una cabecera de tamaño fijo con campos de estado que funcionan como una máquina de estados compacta (in-packet state machine). Los campos ingress_port y egress_port de la cabecera no representan puertos físicos actuales, sino marcadores de estado que permiten al pipeline de egreso de s1 distinguir en qué pasada se encuentra el paquete (camino de ida o de vuelta) sin acceder a tablas adicionales ni a registros stateful. Este patrón es análogo al concepto de medición de latencia en loopback, pero implementado íntegramente en P4 sin hardware especializado [1, 8].

Descripción de la topología

La topología consiste en dos switches P4 y dos hosts en cadena lineal:

- s1 (puerto Thrift 9090): h1 en 10.0.0.1/24 en el puerto 1, s2 en el puerto 2.
- s2 (puerto Thrift 9091): s1 en el puerto 1, h2 en 10.0.0.2/24 en el puerto 2.

Todos los enlaces tienen 5 Mbps de ancho de banda, 5 ms de retardo y 1 % de pérdida de paquetes, configurados mediante TCLink en topo.py. Ambos hosts están en la misma subred /24 con entradas ARP estáticas pre-configuradas por topo.py. El programa P4 implementa dos planos de reenvío paralelos: el tráfico con protocolo $\neq 169$ es manejado por l2_exact_table (reenvío L2 estándar por MAC destino), mientras el tráfico con protocolo = 169 es manejado por TS_table (reenvío contextual por MAC destino más puerto de ingreso). Esta separación permite que ping y iperf funcionen normalmente mientras MySec opera de forma independiente [2].

Cabecera mysec_t y semántica de campos

La cabecera mysec_t ocupa 304 bits (38 bytes) y contiene ocho campos:

```
header mysec_t {  
    bit<4>  ingress_port;  
    bit<4>  egress_port;  
    bit<48> process_time_sw1;  
    bit<48> process_time_sw2;  
    bit<48> egress_time_sw1;  
    bit<48> ingress_back_time_sw1;  
    bit<48> total;  
    bit<48> th;  
}
```

Una distinción crítica en este ejercicio es la diferencia entre mysec.egres_port (campo de la cabecera MySec) y standard_metadata.egress_spec (metadato de BMv2). El primero es un marcador de estado que el programa P4 escribe para comunicar información entre la primera y la segunda pasada de s1; el segundo es el metadato que decide el puerto físico por donde sale el paquete y es fijado por TS_table en ingreso. Modificar mysec.egres_port en egreso no redirige el paquete — el puerto físico ya está determinado por egress_spec — sino que escribe bytes en el header para que la condición de la pasada siguiente pueda distinguir el estado. Esta separación entre estado lógico (en el header) y decisión de reenvío (en metadatos) es un aspecto fundamental del modelo de ejecución de v1model [1, 2].

A pesar de sus nombres, process_time_sw2 no se mide en s2; ambos campos son latencias de s1 medidas en distintas pasadas del mismo switch. s2 actúa exclusivamente como reflector y no escribe ninguna marca de tiempo. El subíndice refleja la dirección en el flujo (sw2 = segundo tramo del camino de ida, sw1 = primer tramo desde la perspectiva del retorno), no el switch físico [8].

Ciclo de vida del paquete MySec

El paquete inicial enviado por `send_mysec.py` porta `ingress_port=1`, `egres_port=2`, y ceros en todos los campos de temporización. El ciclo completo consta de seis etapas:

Etapas A – s1 ingreso (camino de ida, llega en el puerto 1): TS_table coincide con (MAC_h2, `ingress_port=1`) y asigna `egress_spec=2` (hacia s2). La decisión de puerto físico se toma aquí, en ingreso.

Etapas B – s1 egreso (camino de ida): La condición de primera pasada se verifica: `ingress_port==1 && mysec.ingress_port==1 && mysec.egres_port==2`. El pipeline calcula `process_time_sw2 = egress_global_timestamp – ingress_global_timestamp` (latencia de s1 en el tramo de ida). Actualiza la MAC destino Ethernet a la dirección de h1 (para el viaje de vuelta) y actualiza los marcadores de estado a `mysec.ingress_port=1`, `mysec.egres_port=1`.

Etapas C – s2 ingreso (rebote, llega en el puerto 1): TS_table coincide con (MAC_h1, `ingress_port=1`) y asigna `egress_spec=1` (rebota hacia s1). El paquete nunca llega a h2.

Etapas D – s2 egreso (rebote): Ninguna condición de marcado de tiempo se cumple en s2. El switch actúa exclusivamente como reflector y no modifica la cabecera MySec.

Etapas E – s1 ingreso (camino de vuelta, llega en el puerto 2): TS_table coincide con (MAC_h1, `ingress_port=2`) y asigna `egress_spec=1` (hacia h1).

Etapas F – s1 egreso (camino de vuelta): La condición de segunda pasada se verifica: `ingress_port==port1 (=2) && mysec.ingress_port==1 && mysec.egres_port==1`. El pipeline calcula `process_time_sw1 = egress_global_timestamp – ingress_global_timestamp`, registra `egress_time_sw1` e `ingress_back_time_sw1` como timestamps absolutos, calcula `total = process_time_sw1 + process_time_sw2`, y evalúa el indicador `th`. El paquete sale hacia h1 en el puerto 1.

La necesidad de una clave dual (MAC destino + `ingress_port`) en TS_table de s1 se explica por el doble paso: sin el puerto de ingreso como discriminador, s1 no podría distinguir si un paquete MySec con destino MAC_h1 es el de ida

(que todavía no debe ser entregado) o el de vuelta (que sí debe ser entregado a h1). La combinación resuelve esta ambigüedad de forma determinista [8].

Paso 1 – Compilación del programa P4

Una vez completados los TODOs en p4src/main.p4, compilar el programa desde el directorio del ejercicio MySec:

```
mkdir -p p4src/build
```

```
p4c-bm2-ss --p4v 16 -o p4src/build/bmv2.json p4src/main.p4
```

Una compilación sin errores confirma que las expresiones de los TODO son sintácticamente correctas: que los campos de timestamp referenciados existen en standard_metadata, que los tipos de los campos mysec_t son compatibles con las expresiones de diferencia de marcas de tiempo, y que la expresión de umbral produce un tipo bit compatible con el campo th.

Paso 2 – Inicio de la topología Mininet

```
sudo python3 mininet/topo.py
```

La salida observada en la práctica fue la siguiente:

```
*** Creating network
*** Adding hosts:
h1 h2
*** Adding switches:
s1 s2
*** Adding links:
(5.00Mbit 5ms delay 1.00000% loss) (h1, s1)
(5.00Mbit 5ms delay 1.00000% loss) (s1, s2)
(5.00Mbit 5ms delay 1.00000% loss) (s2, h2)
*** Configuring hosts
h1 h2
*** Starting controller

*** Starting 2 switches
s1 Starting s1 @ thrift-port 9090
    cmd: simple_switch -i 1@s1-eth1 -i 2@s1-eth2 --thrift-port 9090
        --device-id 0 ../p4src/build/bmv2.json
s2 Starting s2 @ thrift-port 9091
```

```
cmd: simple_switch -i 1@s2-eth1 -i 2@s2-eth2 --thrift-port 9091
--device-id 1 .../p4src/build/bmv2.json
```

La salida confirma la topología lineal: tres enlaces con las características de calidad configuradas (5 Mbps, 5 ms, 1 % de pérdida) y dos switches arrancados con sus respectivos puertos Thrift. Cada instancia BMv2 tiene únicamente dos puertos de interfaz (uno hacia el host y uno hacia el otro switch), a diferencia del ejercicio MRI donde cada switch gestionaba hasta cuatro interfaces.

Paso 3 – Instalación de las reglas de reenvío

En un segundo terminal, instalar las reglas en ambos switches:

```
simple_switch_CLI --thrift-port 9090 <s1-commands.txt
```

```
Obtaining JSON from switch...
Done
Control utility for runtime P4 table manipulation
RuntimeCmd: Adding entry to exact match table IngressPipeImpl.
  l2_exact_table
match key:      EXACT-00:00:00:00:00:01
action:         IngressPipeImpl.set_egress_port
runtime data:   00:01
Entry has been added with handle 0
RuntimeCmd: Adding entry to exact match table IngressPipeImpl.
  l2_exact_table
match key:      EXACT-00:00:00:00:00:02
action:         IngressPipeImpl.set_egress_port
runtime data:   00:02
Entry has been added with handle 1
RuntimeCmd: Adding entry to exact match table IngressPipeImpl.TS_table
match key:      EXACT-00:00:00:00:00:02    EXACT-00:01
action:         IngressPipeImpl.TimeStamp_port
runtime data:   00:02
Entry has been added with handle 0
RuntimeCmd: Adding entry to exact match table IngressPipeImpl.TS_table
match key:      EXACT-00:00:00:00:00:01    EXACT-00:02
action:         IngressPipeImpl.TimeStamp_port
runtime data:   00:01
Entry has been added with handle 1
```

```
simple_switch_CLI --thrift-port 9091 <s2-commands.txt
```

```
Obtaining JSON from switch...
Done
```



```

Control utility for runtime P4 table manipulation
RuntimeCmd: Adding entry to exact match table IngressPipeImpl.
    l2_exact_table
match key:          EXACT-00:00:00:00:00:01
action:             IngressPipeImpl.set_egress_port
runtime data:       00:01
Entry has been added with handle 0
RuntimeCmd: Adding entry to exact match table IngressPipeImpl.
    l2_exact_table
match key:          EXACT-00:00:00:00:00:02
action:             IngressPipeImpl.set_egress_port
runtime data:       00:02
Entry has been added with handle 1
RuntimeCmd: Adding entry to exact match table IngressPipeImpl.TS_table
match key:          EXACT-00:00:00:00:00:02    EXACT-00:01
action:             IngressPipeImpl.TimeStamp_port
runtime data:       00:01
Entry has been added with handle 0
RuntimeCmd: Adding entry to exact match table IngressPipeImpl.TS_table
match key:          EXACT-00:00:00:00:00:01    EXACT-00:01
action:             IngressPipeImpl.TimeStamp_port
runtime data:       00:01
Entry has been added with handle 1

```

Se instalaron cuatro entradas en total por switch. Las entradas `l2_exact_table` son idénticas en ambos switches: MAC 00:00:00:00:00:01 sale por el puerto 1 y MAC 00:00:00:00:00:02 sale por el puerto 2. Estas entradas permiten que ping funcione correctamente.

Las entradas `TS_table` revelan la lógica del rebote. En s1: la primera entrada (EXACT-00:00:00:00:00:02, EXACT-00:01) dirige paquetes MySec con destino MAC h2 llegando en el puerto 1 hacia el puerto 2 (tramo de ida, $h1 \rightarrow s2$); la segunda entrada (EXACT-00:00:00:00:00:01, EXACT-00:02) dirige paquetes MySec con destino MAC h1 llegando en el puerto 2 hacia el puerto 1 (tramo de vuelta, $s2 \rightarrow h1$). En s2, ambas entradas dirigen el tráfico hacia el puerto 1 (de regreso a s1). La segunda entrada de s2 (EXACT-00:00:00:00:00:01, EXACT-00:01) intercepta el paquete rebotado cuya MAC destino es ya la de h1 y llega por el puerto 1 de s2, devolviéndolo a s1 sin entregarlo a h2. Esta asimetría entre s1 y s2 implementa el rebote en el plano de control [2].

Paso 4 – Verificación de conectividad

```
mininet> pingall
```

```
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
```

Ambos hosts se alcanzan con 0 % de pérdida. La conectividad estándar está operativa. A diferencia del ejercicio MRI donde los pares en la misma subred fallaban por la ausencia de manejo ARP, aquí no existe ese problema porque ambos hosts están en la misma subred /24 con entradas ARP estáticas pre-configuradas, y `I2_exact_table` reenvía el tráfico ICMP correctamente.

Paso 5 – Prueba MySec y resultados

Enviar un paquete MySec desde h1 y observar el resultado:

```
mininet> h1 python3 mininet/send_mysec.py
```

La salida observada en la práctica fue la siguiente:

```
Enviando paquete MySec por eth0...
###[ Ethernet ]###
  dst      = 00:00:00:00:00:02
  src      = 00:00:00:00:00:01
  type     = IPv4
###[ IP ]###
  version  = 4
  ihl      = 5
  tos      = 0x0
  len      = 57
  id       = 1
  flags    =
  frag     = 0
  ttl      = 64
  proto    = 169
  chksum   = 0x6619
  src      = 10.0.0.1
  dst      = 10.0.0.2
###[ MySec ]###
  ingress_port= 1
  egress_port= 2
  process_time_sw1= 0
```

```

    process_time_sw2= 0
    egress_time_sw1= 0
    ingress_back_time_sw1= 0
    total          = 0
    th             = 0

Paquete recibido:
###[ Ethernet ]###
    dst          = 00:00:00:00:00:01
    src          = 00:00:00:00:00:01
    type         = IPv4
###[ IP ]###
    version      = 4
    ihl          = 5
    tos          = 0x0
    len          = 57
    id           = 1
    flags        =
    frag         = 0
    ttl          = 64
    proto        = 169
    chksum       = 0x6619
    src          = 10.0.0.1
    dst          = 10.0.0.2
###[ MySec ]###
    ingress_port= 2
    egress_port= 1
    process_time_sw1= 630
    process_time_sw2= 679
    egress_time_sw1= 13149994
    ingress_back_time_sw1= 13149364
    total        = 1309
    th           = 0

=====
MySec -- Resultados de latencia in-switch
=====
process_time_sw1      : 630 ns  (latencia egress S1, camino vuelta s2->
    h1)
process_time_sw2      : 679 ns  (latencia egress S1, camino ida h1->s2)
egress_time_sw1       : 13149994 (timestamp egress S1)
ingress_back_time_sw1 : 13149364
total                 : 1309
th                    : 0
=====

Timestamps registrados correctamente

```

El paquete enviado tiene todos los campos de temporización en cero, con in-

gress_port=1 y egres_port=2 como marcadores de estado iniciales. El paquete recibido de retorno muestra los timestamps completados correctamente. El análisis de cada campo es el siguiente:

process_time_sw2 = 679 μ s: Latencia de egreso de s1 en el tramo de ida ($h1 \rightarrow s2$). Calculada como `egress_global_timestamp - ingress_global_timestamp` durante la primera pasada de s1 (Etapa B). El valor refleja el tiempo que el paquete permaneció en s1 entre su llegada y el inicio de su transmisión de egreso.

process_time_sw1 = 630 μ s: Latencia de egreso de s1 en el tramo de vuelta ($s2 \rightarrow h1$). Calculada durante la segunda pasada de s1 (Etapa F). El valor ligeramente menor al del tramo de ida refleja la variabilidad intrínseca del switch software (BMv2).

egress_time_sw1 = 13149994 e ingress_back_time_sw1 = 13149364: Marcas de tiempo absolutas en microsegundos (desde el arranque del switch) del pipeline de egreso e ingreso de s1 en el tramo de vuelta. La diferencia $13149994 - 13149364 = 630$ coincide exactamente con `process_time_sw1`, validando la correcta implementación de los campos. El valor absoluto de $13\,149\,994 \mu s$ ($\approx 13,15$ s) es coherente con el tiempo transcurrido desde el arranque del switch software hasta la ejecución de la prueba.

total = 1309 μ s: Suma de las dos latencias ($630 + 679 = 1309 \mu s$). Ambos valores son del orden de cientos de microsegundos, muy por debajo de los $5.000 \mu s$ (5 ms) del retardo de enlace configurado, confirmando que los valores miden el procesamiento del switch y no la propagación en el enlace. Nota: el script `send_mysec.py` etiqueta estos valores con la unidad “ns”; la unidad correcta en BMv2 simple_switch es microsegundos (μs).

th = 0: El total acumulado ($1309 \mu s$) no supera el umbral de $5000 \mu s$ configurado en el código, por lo que el indicador permanece en 0.

Los marcadores de estado cambiaron de `ingress_port=1, egres_port=2` a `ingress_port=2, egres_port=1`, reflejando el estado al final del recorrido completo. En el paquete retornado, la MAC Ethernet destino es `00:00:00:00:00:01` ($h1$) porque la acción de Etapa B reescribió el campo `dst_addr` para preparar

el camino de vuelta. La MAC fuente permanece 00:00:00:00:00:01 porque el protocolo MySec no la modifica. El campo ihl = 5 confirma que el encabezado IPv4 no creció durante el tránsito, a diferencia de MRI: el encabezado my-sec.t es transportado como carga de protocolo IP 169, no como opción IPv4 [1, 2, 8].

Análisis del pipeline P4

Tabla TS_table — clave de dos campos

A diferencia de l2_exact_table que usa un solo campo de coincidencia (MAC destino), TS_table usa dos claves simultáneas: MAC destino e ingress_port. El control de ingreso de IngressPipeImpl aplica TS_table solo cuando el protocolo IP es 169; de lo contrario aplica l2_exact_table. Esta separación garantiza que el tráfico estándar nunca active la lógica MySec.

Pipeline de egreso — condiciones de tramo

El control de egreso EgressPipeImpl establece local_metadata.port1 = 2 y evalúa dos condiciones mutuamente excluyentes:

```
// Condicion de vuelta (segunda pasada de s1):
if (standard_metadata.ingress_port == local_metadata.port1
    && hdr.mysec.ingress_port == 1
    && hdr.mysec.egres_port == 1) {
    hdr.mysec.process_time_sw1 =
        egress_global_timestamp - ingress_global_timestamp;
    hdr.mysec.egress_time_sw1 = egress_global_timestamp;
    hdr.mysec.ingress_back_time_sw1 = ingress_global_timestamp;
    hdr.mysec.total = process_time_sw1 + process_time_sw2;
    // ... evaluacion de umbral ...
}

// Condicion de ida (primera pasada de s1):
else if (standard_metadata.ingress_port == 1
    && hdr.mysec.ingress_port == 1
    && hdr.mysec.egres_port == 2) {
    hdr.mysec.process_time_sw2 =
```

```
egress_global_timestamp - ingress_global_timestamp;  
hdr.ethernet.dst_addr = 0x000000000001; // MAC de h1  
hdr.mysec.ingress_port = 1; hdr.mysec.egres_port = 1;  
}
```

Los metadatos `egress_global_timestamp` e `ingress_global_timestamp` de la arquitectura v1model se usan en el pipeline de egreso, razón por la que toda la lógica de temporización opera allí. En ningún switch (s1 ni s2) se realiza el cálculo de latencia en el pipeline de ingreso porque `egress_global_timestamp` no está disponible hasta el inicio del procesamiento de egreso; es en ese punto donde ambos timestamps son simultáneamente coherentes y su diferencia es significativa [1, 2].

Discusión comparativa

Tipo de telemetría: trazas de ruta versus latencia de procesamiento

MRI y MySec representan dos filosofías de telemetría en banda complementarias. MRI implementa un modelo acumulativo y distribuido: cada switch en el camino contribuye con su propio registro, y el receptor final obtiene una vista completa del trayecto y el estado de congestión en cada nodo. La información es multidimensional (camino + profundidad de cola por salto) y útil para diagnóstico de routing y detección de cuellos de botella en tiempo real. MySec implementa un modelo focalizado y reflexivo: un único switch instrumentado (s1) registra su propia latencia en dos momentos diferentes del ciclo de vida del paquete, usando el camino de rebote como mecanismo de recuperación de datos sin coordinar con un colector externo. MRI es distribuido (requiere la cooperación de todos los switches en el camino); MySec es centralizado en un único switch que procesa el paquete dos veces [4, 9].

Mecanismo de señalización del protocolo

MRI usa un número de opción IPv4 experimental (Option Number = 31 en el campo de 5 bits del tipo de opción), valor no asignado por IANA, como señal

de presencia en el paquete; el esquema de codificación de opciones (copy-Flag — optClass — option) sí está definido en RFC 791 [5]. MySec usa el número de protocolo IP 169, no registrado en IANA. Esta elección es deliberada para el contexto educativo: el número actúa como selector en el parser P4 para bifurcar hacia el estado de extracción de mysec.t. En un entorno de producción, el uso de números de protocolo no estándar genera problemas de interoperabilidad con middleboxes. La alternativa de producción para ambos mecanismos es INT sobre encabezados encapsulados (por ejemplo, INT sobre VXLAN con GRE) que transportan datos de telemetría sin modificar las cabeceras IP visibles a los equipos intermedios [4].

Punto del pipeline donde ocurre la medición

Ambos ejercicios realizan sus mediciones en el pipeline de egreso. En MRI, la razón es que `standard_metadata.deq_qdepth` solo está disponible en egreso: captura el estado de la cola cuando el paquete fue retirado para transmisión. En MySec, los timestamps de `v1model` solo son significativos en egreso porque es el primer punto donde tanto `egress_global_timestamp` como `ingress_global_timestamp` son simultáneamente coherentes para la misma instancia de procesamiento del paquete [1, 2].

MRI modifica los campos `ihl`, `totalLen` y `optionLength` del encabezado IPv4 en cada salto, y por lo tanto requiere que `MyComputeChecksum` recalculé el checksum IPv4. MySec no modifica ningún campo del encabezado IPv4 estándar (solo escribe en la cabecera `mysec.t` que sigue al encabezado IP como protocolo 169), por lo que su bloque de checksum puede dejarse vacío sin afectar la integridad del encabezado IP. Esto explica que `ihl` sea 5 en los paquetes MySec y 10 en los paquetes MRI [5, 1].

Escalabilidad y limitaciones

MRI está acotado por `MAX_HOPS=9` y por el campo `ihl` de 4 bits (máximo 60 bytes de encabezado IPv4). En una red de muchos saltos, las opciones alcanzarían el límite de tamaño del encabezado. Adicionalmente, cada registro

agrega 8 bytes al paquete, lo que puede superar el MTU en rutas largas. El parser recursivo añade complejidad de estado que crece linealmente con el número de registros; en hardware real, el número de llamadas al parser está acotado por la profundidad máxima del pipeline [4, 9].

MySec está acotado por la estructura de su cabecera fija de 38 bytes: puede medir exactamente dos instancias de procesamiento de un mismo switch. Extenderlo a múltiples switches requeriría un stack de cabeceras análogo al de MRI, o un protocolo de recolección fuera de banda. La precisión de medición también está influenciada por la variabilidad del planificador de la máquina virtual anfitriona (BMv2 es software de propósito educativo, no un ASIC de producción).

Conclusión

A partir de las capturas y mediciones presentadas, se verificó empíricamente que el pipeline de egreso puede instrumentarse para registrar, por paquete, la ruta y la congestión por salto (MRI), así como la latencia de procesamiento de un nodo concreto (MySec), sin intervención del plano de control durante la fase de medición. En particular, el ejercicio MRI mostró cómo un conjunto de switches puede construir colectivamente un registro de la trayectoria y del estado de congestión directamente dentro del paquete de datos. La observación del crecimiento de la pila swtrace (count=2, swid variables según la ruta) y la variación de los valores qdepth bajo congestión inducida por iperf (qdepth = 61 para swid=1 durante saturación del enlace cuello de botella) validan que el pipeline de egreso opera correctamente.

El ejercicio MySec demostró cómo un protocolo de medición de latencia puede implementarse en el plano de datos usando el camino de rebote para eliminar la necesidad de sincronización de relojes entre switches. Los valores obtenidos (process_time_sw1 = 630 μ s, process_time_sw2 = 679 μ s, total = 1309 μ s) son consistentes con el comportamiento esperado de BMv2 en una máquina virtual, y la coherencia entre egress_time_sw1 (13149994) e ingress_back_time_sw1 (13149364) cuya diferencia exacta coincide con pro-

cess_time_sw1 valida la correcta implementación de los campos [1].

El flujo de trabajo de compilación e implantación (p4c-bm2-ss, simple_switch, simple_switch_CLI) fue idéntico al de los laboratorios anteriores, con la adición del comando table_set_default para tablas sin clave de coincidencia, que es la contribución metodológica distintiva del ejercicio MRI. Juntos, los dos ejercicios ilustran que el plano de datos P4 puede actuar no solo como reenviador de paquetes sino como instrumento de medición de la propia red, siendo esta la base conceptual de INT completo y de los sistemas de detección de anomalías embebidos en el plano de datos [6, 4, 9].

Respuestas a las preguntas de discusión

Pregunta 1. ¿Por qué solo MRI requiere recálculo activo del checksum IPv4?

Solo MRI requiere recálculo activo del checksum IPv4. La acción add_swtrace modifica ihl y totalLen en cada salto, invalidando el checksum calculado por el emisor original. En MySec, ningún campo del encabezado IPv4 estándar es modificado; la cabecera mysec_t es transportada como carga del protocolo 169 y no forma parte del cálculo del checksum IPv4. El checksum original de h1 sigue siendo válido a lo largo de todo el camino de rebote, lo que también explica por qué ihl = 5 es constante en MySec [5, 1].

Pregunta 2. ¿Por qué tanto deq_qdepth (MRI) como los timestamps de v1model (MySec) deben leerse en el pipeline de egreso?

Ambos metadatos representan el estado del switch en el momento de la transmisión, posterior al procesamiento de ingreso. deq_qdepth mide la profundidad de la cola cuando el paquete fue retirado; esta operación de dequeue ocurre entre ingreso y egreso. Los timestamps tienen sus valores finales en egreso: es el primer punto donde tanto egress_global_timestamp como ingress_global_timestamp son coherentes para el tránsito actual [1, 2].

Pregunta 3. ¿Qué modificación en las tablas de reenvío permitiría observar count = 3 en la pila MRI?

Para obtener count=3 sería necesario configurar una ruta que atravesara los

tres switches consecutivamente ($s1 \rightarrow s2 \rightarrow s3 \rightarrow h3$). Esto requeriría modificar la entrada LPM de $s1$ para el prefijo 10.0.3.0/24 a fin de reenviar hacia $s2$ (puerto 3) en lugar de $s3$ (puerto 4), y añadir en $s2$ la ruta correspondiente hacia $s3$. Esta configuración forzaría los paquetes $h1 \rightarrow h3$ por los tres switches, de modo que el receptor observaría tres registros en la pila en orden inverso al camino: {swid=3, swid=2, swid=1} [3].

Pregunta 4. ¿Cómo se implementaría INT en producción para evitar los problemas de interoperabilidad de MRI y MySec?

El mecanismo estándar equivalente es INT en modo INT-XD (eXport Data) con Telemetry Report: los datos de telemetría se exportan como paquetes UDP separados hacia un colector, sin modificar el paquete original. Esto garantiza compatibilidad total con routers y firewalls estándar. Para redes IPv6, P4.org estandarizó el uso de encabezados Hop-by-Hop Options como transporte nativo de metadatos INT. Sistemas completos como los descritos por Garzón et al. integran los informes con colectores InfluxDB/Grafana para visualización en tiempo real [4, 9].

Pregunta 5. ¿Cuáles son las limitaciones de escalar MRI y MySec a redes de muchos saltos?

En MRI, 10 saltos producirían 80 bytes de opciones IPv4, pero el límite del campo ihl (4 bits, máximo 60 bytes de encabezado) solo permite aproximadamente 6 registros adicionales sobre el mínimo de 20 bytes de opciones base. MRI está fundamentalmente limitado por el estándar IPv4 a menos que se use un mecanismo de transporte diferente. En MySec, la sobrecarga es fija (38 bytes), pero escalar a múltiples switches de medición requeriría múltiples cabeceras MySec anidadas o un stack análogo al de MRI, aumentando la complejidad del parser y el tamaño del paquete. En ambos casos, la exactitud de las mediciones se degrada en entornos de alta variabilidad (máquinas virtuales con hipervisores, planificadores de CPU compartidos) [4, 9].

Referencias

- [1] P4.org, “P4₁₆ Language Specification,” ver. 1.2.5, 2024. [En línea]. Disponible en: <https://p4.org/p4-spec>
- [2] P4.org, “Behavioral Model (BMv2) Documentation,” 2020–2024. [En línea]. Disponible en: <https://github.com/p4lang/behavioral-model>
- [3] P4.org, “P4 Language Tutorials,” repositorio oficial de ejercicios de referencia, 2024. [En línea]. Disponible en: <https://github.com/p4lang/tutorials>
- [4] P4.org, “In-Band Network Telemetry (INT) Dataplane Specification,” ver. 2.1, 2020. [En línea]. Disponible en la sección de especificaciones de P4.org: <https://p4.org/specs>
- [5] J. Postel, “Internet Protocol,” RFC 791, IETF, Sep. 1981.
- [6] C. Kim et al., “In-band Network Telemetry via Programmable Dataplanes,” presentación en ACM SIGCOMM, 2015. [En línea]. Disponible en: <https://p4.org/assets/INT-current-spec.pdf>
- [7] J. F. Botero, “Data Plane Programmability — Introduction and Applications,” Univ. de Antioquia / Univ. of Antwerp, notas de curso Internet del Futuro, 2020.
- [8] J. F. Botero, “Data Plane Programmability — P4 Language,” Univ. de Antioquia / Univ. of Antwerp, notas de curso Internet del Futuro, 2020.
- [9] C. Garzón et al., “P4 Cybersecurity Solutions: Taxonomy and Open Challenges,” IEEE Access, vol. 12, 2024.
- [10] P. Bosshart et al., “P4: Programming Protocol-Independent Packet Processors,” ACM SIGCOMM Computer Communication Review, vol. 44, núm. 3, pp. 87–95, 2014.